Day-11 - 9/3/26

Why Networking is Important in kubernetes

Pod to Pod communi
↳ like CNI o

Containers are dynamic & short lived

Pods can be created and destroyed anytime

Appi must commu reliably

It provides a cluster-wide networking model.

Kubernetes Networking Model

Each pods gets a unique Ip address

Pods comm directly w/o NAT

Nodes can comm with pods

Containers in the same pods share same network namespace

CNI → Container Network Interface

It is a standard used to configure networking for containers.

It assign Ip address to pods

It connects pods to the cluster network

Ex: Calico, Flannel, Cilium

Scanned with OKEN Scanner

# Kubernetes Service

A Service provides a stable network endpoint for pods. Pods are temp & their IPs change.

Service provide a fixed IP & DNS name

They load balance traffic across multiple pods

## Types

Cluster IP → Internal commun within cluster → default IP

Nodeport → Exposes service on node IP & Port → External IP ku

Load balancer → Cloud provide external load balan → via traffic assign pdhnum - eg

External name → Maps service to external DNS
↳ DNS name resolution

## Role of Kube-Proxy

- Runs on every node
- Watches service & endpoint
- Create iptables or IPVS rules
- Implement service virtual IP routing

Scanned with OKEN Scanner

## Understanding Kubernetes Objects :-

Kubernetes object are persistent entities in the kubernetes sys. It uses these entities to represent the state of your cluster.

Describe

what containerized appli are running (& on which nodes). The resources avai to those appli. The policies around how those appli behave, such as restart policies, upgrade & fault tolerance.

It is a record of intent that defines the desired state of your cluster. Once created continously monitors & ensure the object exist & matches that desired state.

To create, update or delete these object you use the kubernetes API. The kubectl command-line tool communi with API for you. Can directly use cli lib to interact with API in own programs

## Deployment Resource Object :-

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 2
  template:
```

```
metadata:
  labels:
    app: nginx

spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerport: 80
```

Pod Resource Object

```
apiversion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerport: 80
```

Defining a Service

A Service is an Object (the same way that a Pod or a configmap is an Object). You can create view or modify service definitions using kubernets API.

Scanned with OKEN Scanner

apiVersion: v1
kind: Service
metadata:
  name: my.service

spec:
  Selector
    app.kubernetes.io/name: MyApp
  Ports:
    protocol: TCP
    port: 80
    targetPort: 9376

Commands:-

1) kubectl version
2) kubectl get nodes
3) kubectl describe node ___ (nodename) => metadata of node
4) kubectl run sitpod (pod name) --image=httpd
5) kubectl get pods
6) kubectl describe pod sitpod (podname)
7) kubectl exec -it sitpod -- /bin/bash (p.name)
8) kubectl logs sitpod (p.name)
9) kubectl delete pod sitpod (podname)
10) kubectl get pods

Deployment based Commands

11) kubectl create deployment sitdeploy --image=httpd:latest (scaling, self healing)
12) kubectl get deployment
13) kubectl get pods
14) kubectl delete pod (pod name) ↯ using deployment it cannot delete pod next pod are being create